CASCADIAJS 2026

Jonathan Keslin · Atlassian

Shared Components Beyond the Design System

Jonathan Keslin

Senior Principal Engineer
Navigation & Admin

ATLASSIAN

PREVIOUSLY AT

GoDaddy

Microsoft

CASCADIAJS 2026

Jonathan Keslin · Atlassian

Shared Components Beyond the Design System

What's a Design System?

UX foundations
Defines the look and feel for a product or company

Atomic components
Often basic components like buttons, dropdowns, modals

Primitives & tokens
Includes primitives like the color palette, spacing system, and typography

But a design system can’t do everything

Beyond the design system

Navigation &
higher-level UX

Team-specific
components

Design system
candidates

Shared Components **Beyond** the Design System

Where we’re headed

01

Who are you building for?

02

Where are you building it?

03

Building it thoughtfully

04

Keeping it maintainable

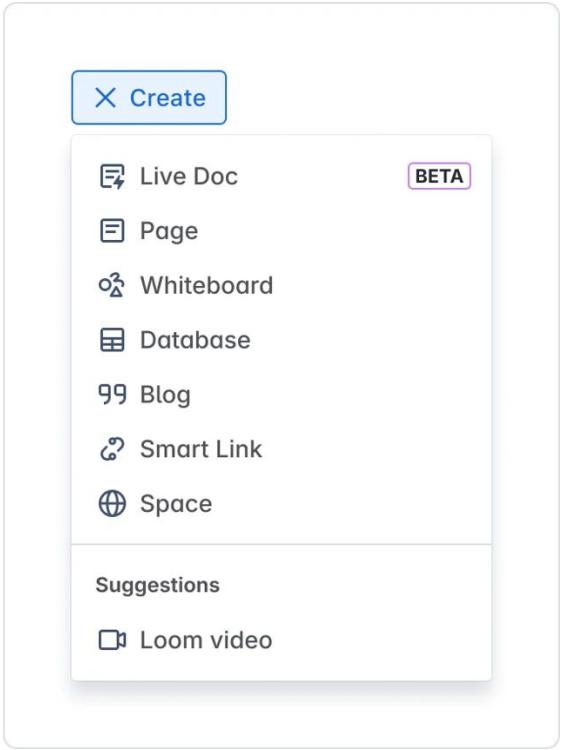
05

Fostering an ecosystem

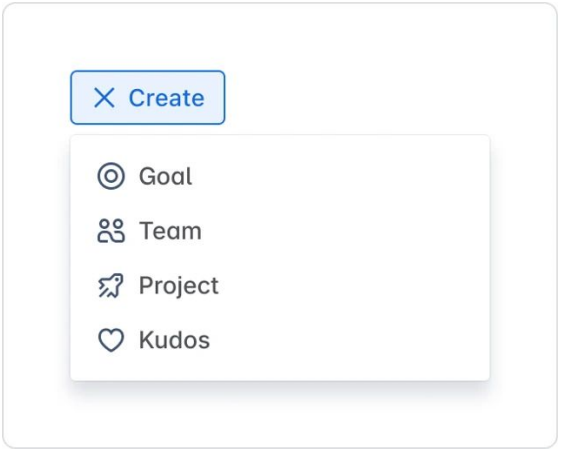
Who are you building for?

Anchor consumer
The team for whom you're building right now — might even be your team

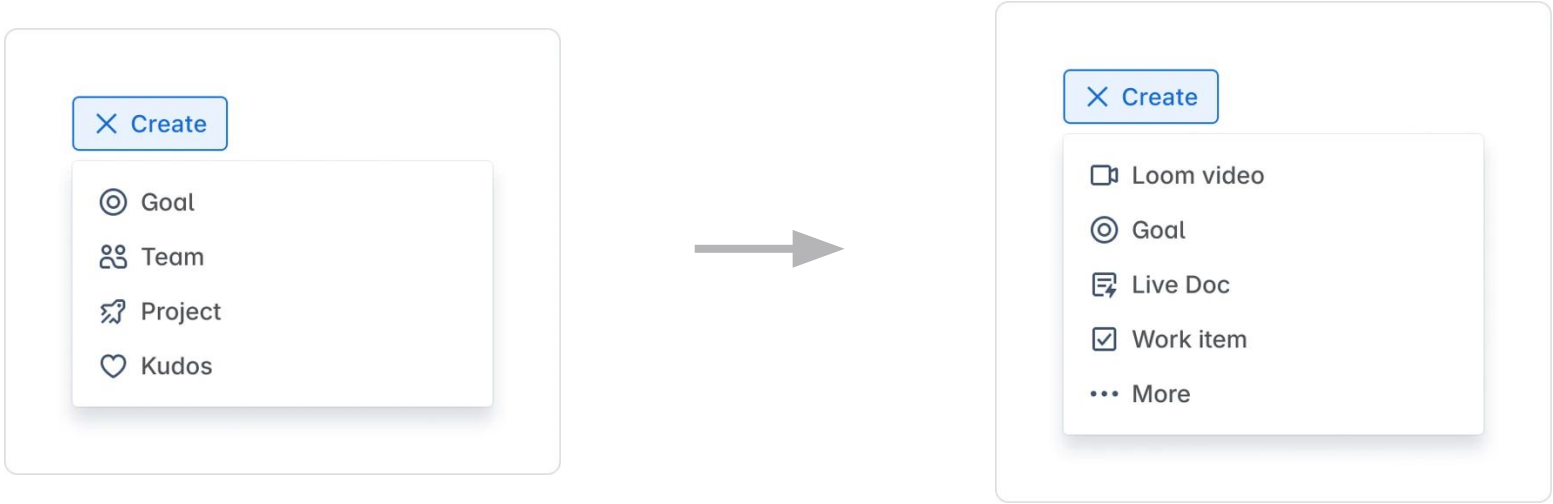
Second consumer
A real or imaginary second team — to ensure you keep your component generic



Confluence



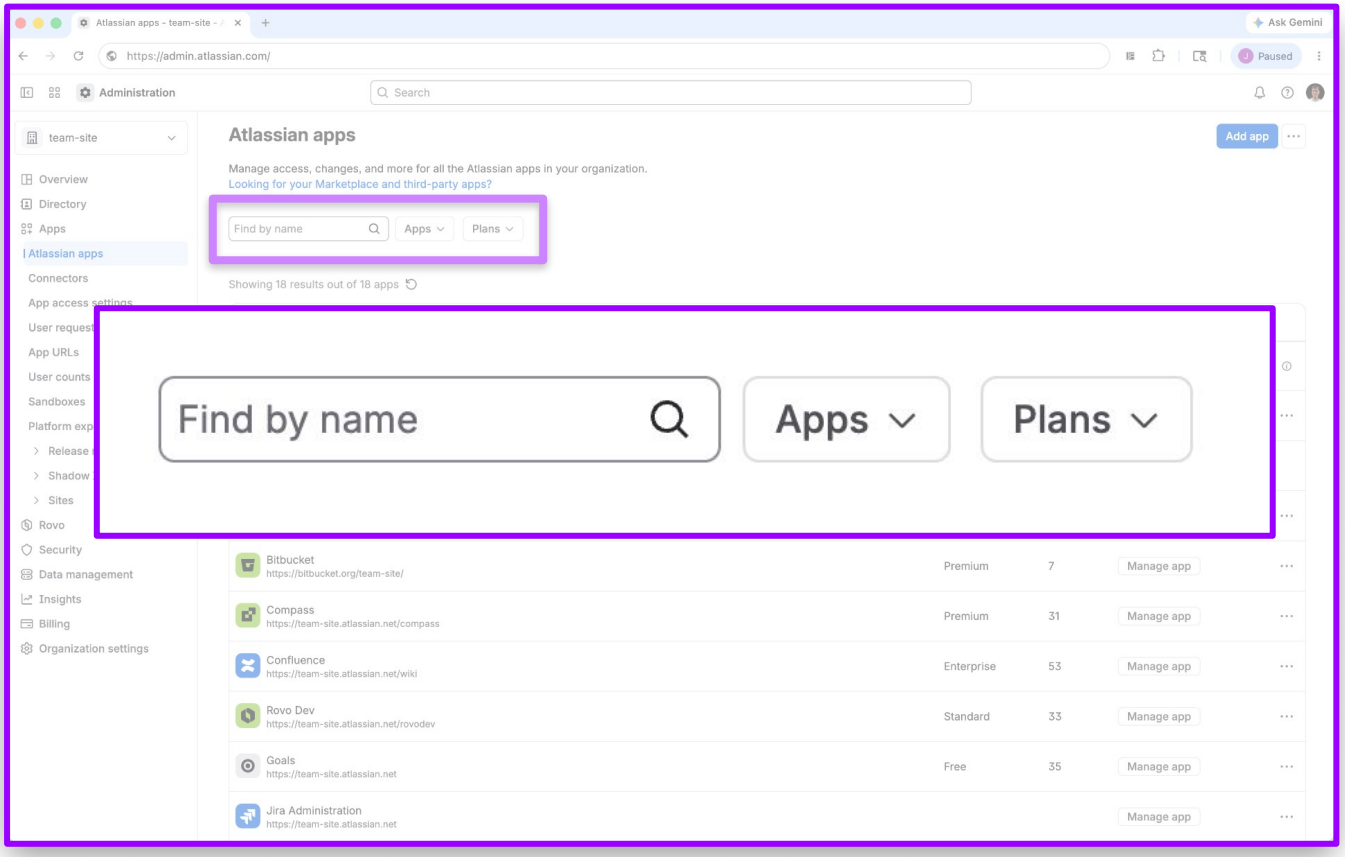
Home


Create
Goal
Team
Project
Kudos
Create
Loom video
Goal
Live Doc
Work item
More

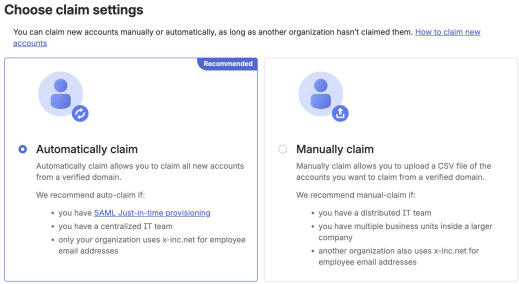
Where are you building it?

Keep it separate
Don’t put it in your app repo — keep it separate so it can evolve independently

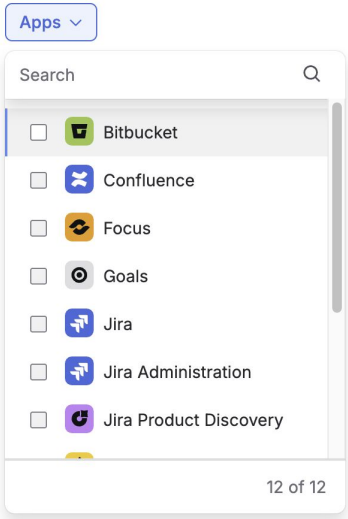
Publish artifacts
Release to npm/artifactory/etc so other teams can use your component freely


Atlassian apps - team-site -
Ask Gemini
https://admin.atlassian.com/
Paused
Administration
Search
team-site
Overview
Directory
Apps
Atlassian apps
Connectors
App access settings
User requests
App URLs
User counts
Sandboxes
Platform exp
Release
Shadow
Sites
Rovo
Security
Data management
Insights
Billing
Organization settings
Atlassian apps
Add app
Manage access, changes, and more for all the Atlassian apps in your organization.
Looking for your Marketplace and third-party apps?
Find by name
Apps
Plans
Showing 18 results out of 18 apps
Find by name
Apps
Plans
Bitbucket
https://bitbucket.org/team-site/
Premium
7
Manage app
Compass
https://team-site.atlassian.net/compass
Premium
31
Manage app
Confluence
https://team-site.atlassian.net/wiki
Enterprise
53
Manage app
Rovo Dev
https://team-site.atlassian.net/rovodev
Standard
33
Manage app
Goals
https://team-site.atlassian.net
Free
35
Manage app
Jira Administration
https://team-site.atlassian.net
Manage app

Filter controller



Select list

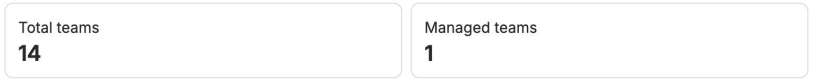


Paginated filter

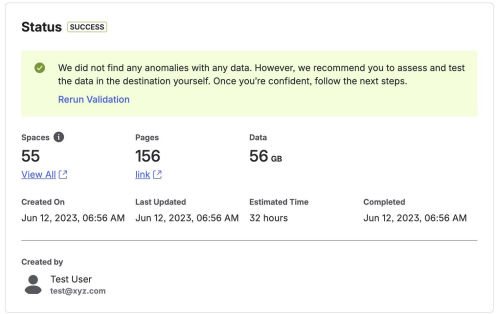
Admin UI Kit

and more...

Pagination



Summary group

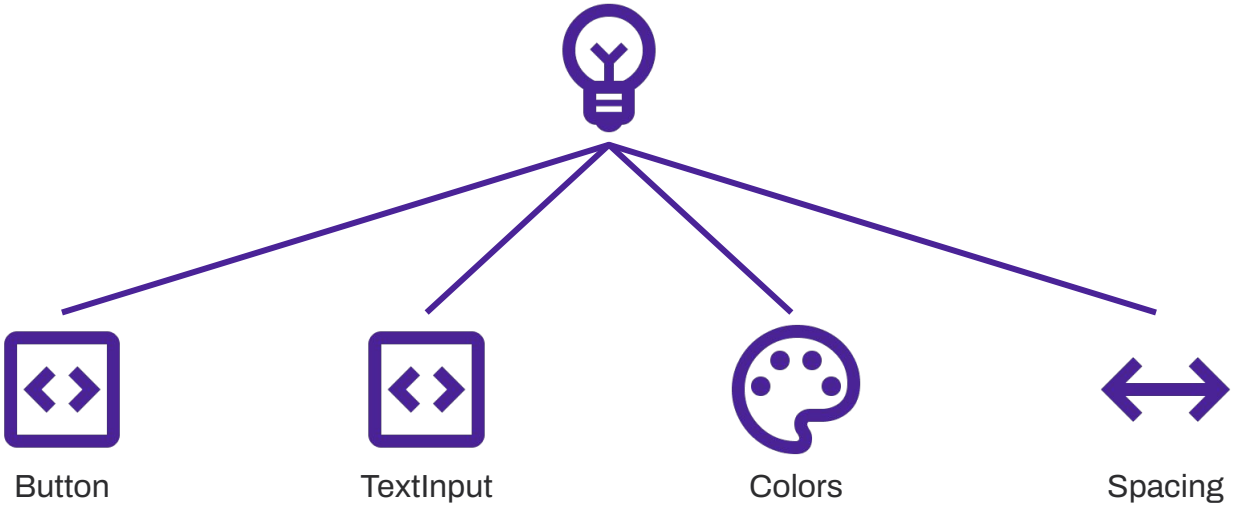


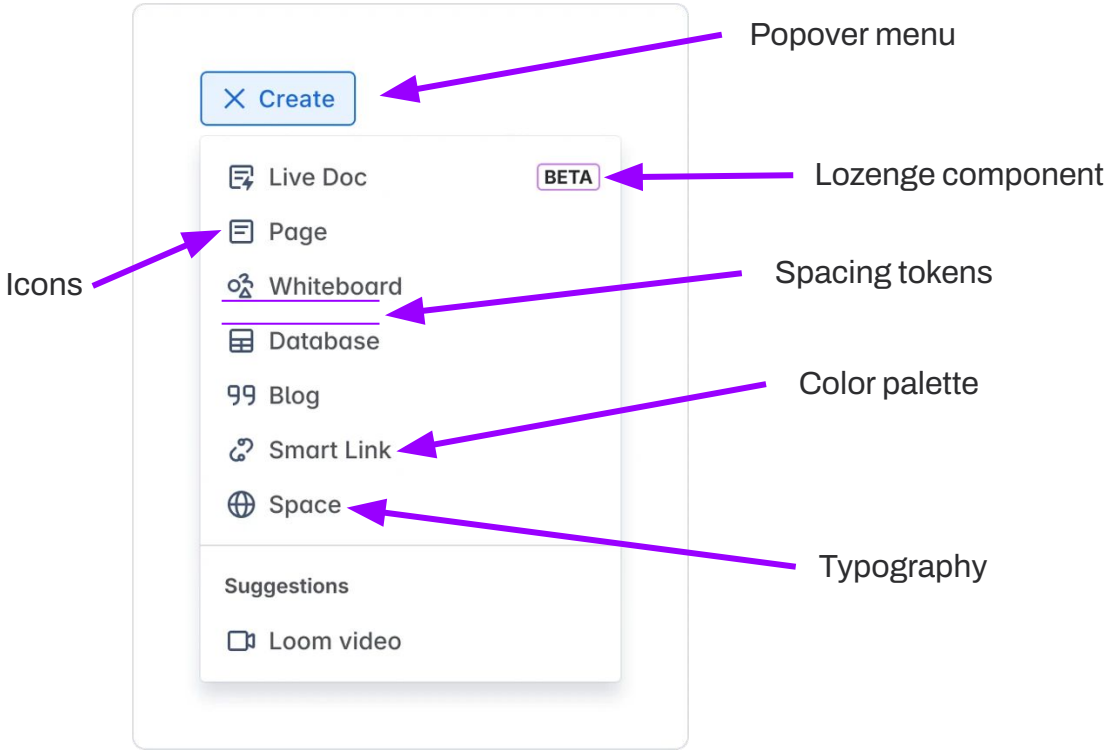
Summary card

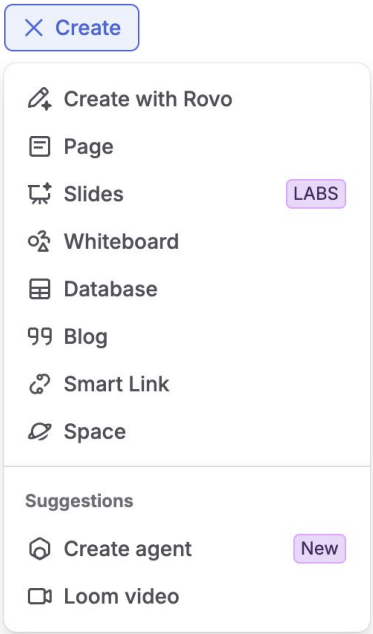
Building it **thoughtfully**

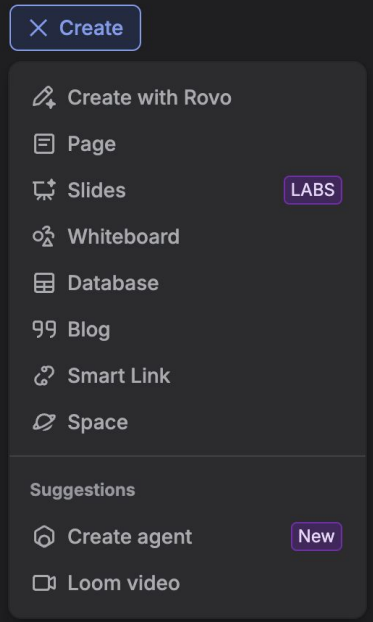
6 principles to follow

Principle 1: Build *up* from the design system




Create
Live Doc
BETA
Page
Whiteboard
Database
Blog
Smart Link
Space
Suggestions
Loom video
Popover menu
Lozenge component
Icons
Spacing tokens
Color palette
Typography


Create
Create with Rovo
Page
Slides
LABS
Whiteboard
Database
Blog
Smart Link
Space
Suggestions
Create agent
New
Loom video


Create
Create with Rovo
Page
Slides
LABS
Whiteboard
Database
Blog
Smart Link
Space
Suggestions
Create agent
New
Loom video

Make your APIs consistent

BASIC TEXT INPUT

```
value: string;
onChange: (newValue: string) => void;
size: “small” | “default” | “large”;
placeholder: string;
maxLength: number;
...
```

SNAZZY SEARCH BOX

```
datasourceUrl: string;
maxOptions: number;
onSubmit: (value: string) => void;
value: string;
onChange: (newValue: string) => void;
size: “small” | “default” | “large”;
placeholder: string;
maxLength: number;
...
```

Principle 2:

Design with **Layered Extensibility**

Make the right way the easy way
Optimize the API for the 80% case, providing sensible defaults where possible

Offer layers of deeper customization
Don’t make consumers eject into hard mode just to get a slightly custom experience

Design with Layered Extensibility

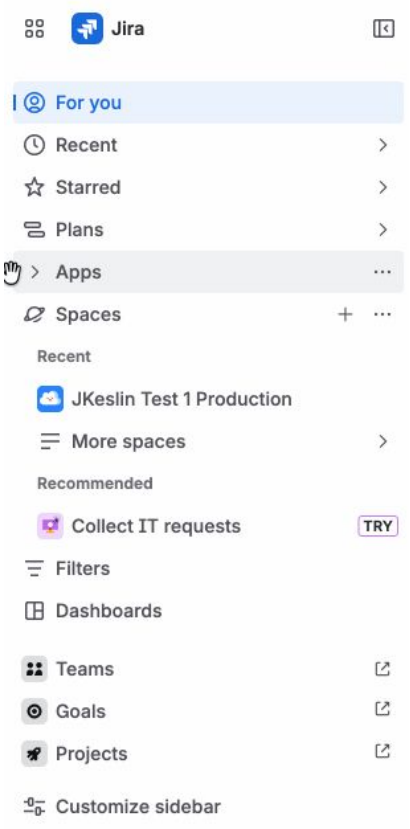
80% CASE: TEXT


```
<Button
  label="My Awesome Button"
/>
```

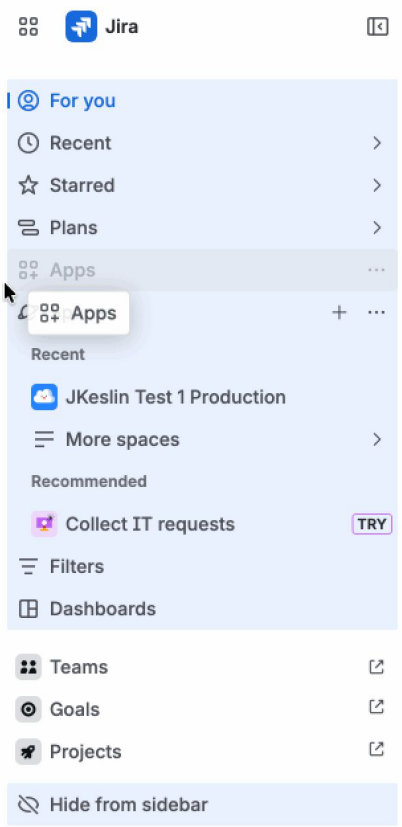
15% CASE: REACT NODE


```
<Button
  label="My Awesome Button">
  <HStack>
    <Icon type="star" />
    My Awesome Button
  </HStack>
</Button>
```

Example: Jira's navigation

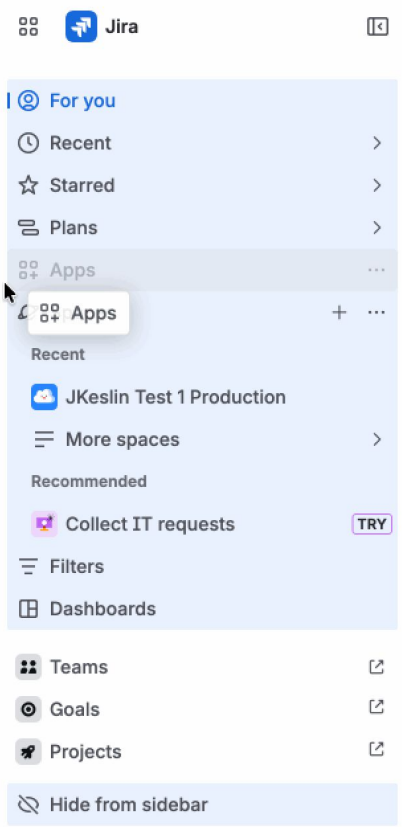


Example: Jira's navigation



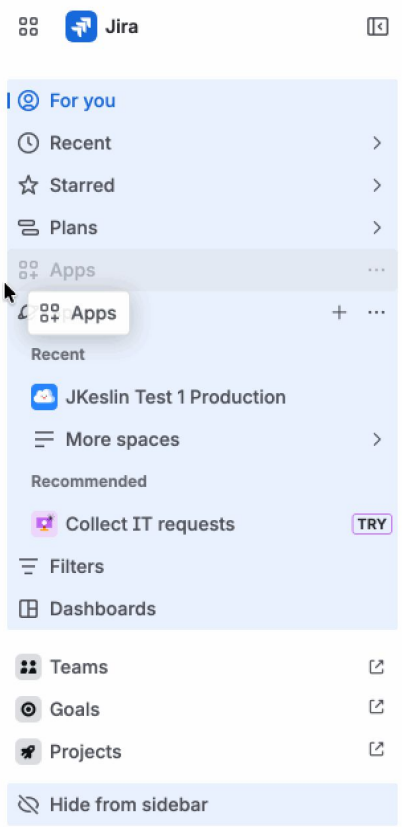

```
// Common case - fully declarative
{
  type: “link”,
  label: “For you”,
  icon: <YouIcon />,
  href: “/for-you”
}
```

Example: Jira's navigation




```
// Layer 2 - passing a React node
{
  type: “link”,
  label: “Apps”,
  labelElem: <AppsLabel />,
  icon: <AppsIcon />,
  href: “/apps”
}
```

Example: Jira's navigation

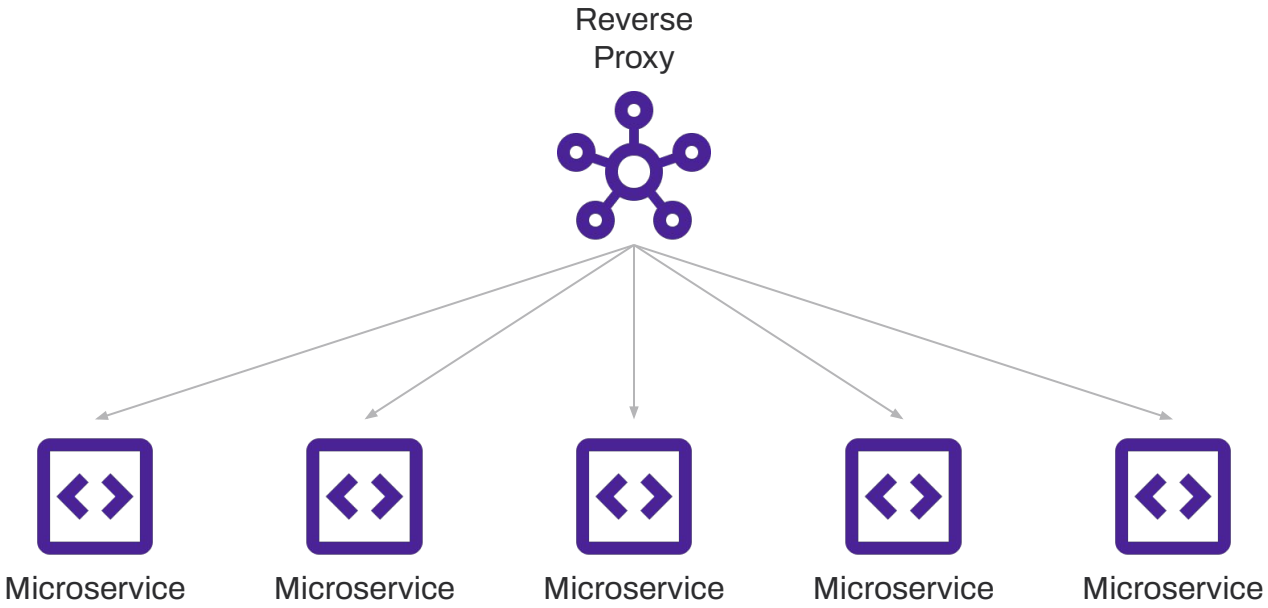



```
// Custom items - functional
{
  type: “custom”,
  label: “Apps”,
  icon: <AppsIcon />,
  component: AppsNavItem
}
```

```
// AppsNavItem props
{
  draggableRef: ReactRef,
  dropIndicator: ReactNode,
  isDragging: boolean,
  ...
}
```

Design with Layered Extensibility

Example: Reverse proxy



Example: Reverse proxy

```
// Layer 1 - simple case
{
  service: “ms1”,
  url: “http://ms1.apps.svc.cluster.local”
}
```

```
// Layer 2 - parameter transformations
{
  service: “ms2”,
  url: “http://ms2.apps.svc.cluster.local”,
  paramTransforms: {
    locale: (loc) =>
      loc.replaceAll(“-”, “_”)
  }
}
```

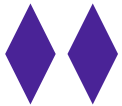
```
// Layer 3 - response transformations
{
  service: “ms3”,
  url: “http://ms3.apps.svc.cluster.local”,
  responseTransform: (res) => {
    if (res.body.error) {
      return {
        status: 500,
        body: { error: res.body.error }
      };
    }
    return {
      status: 200,
      body: transformMS3(res.body)
    };
  }
}
```

This is a simplified example of how this worked.

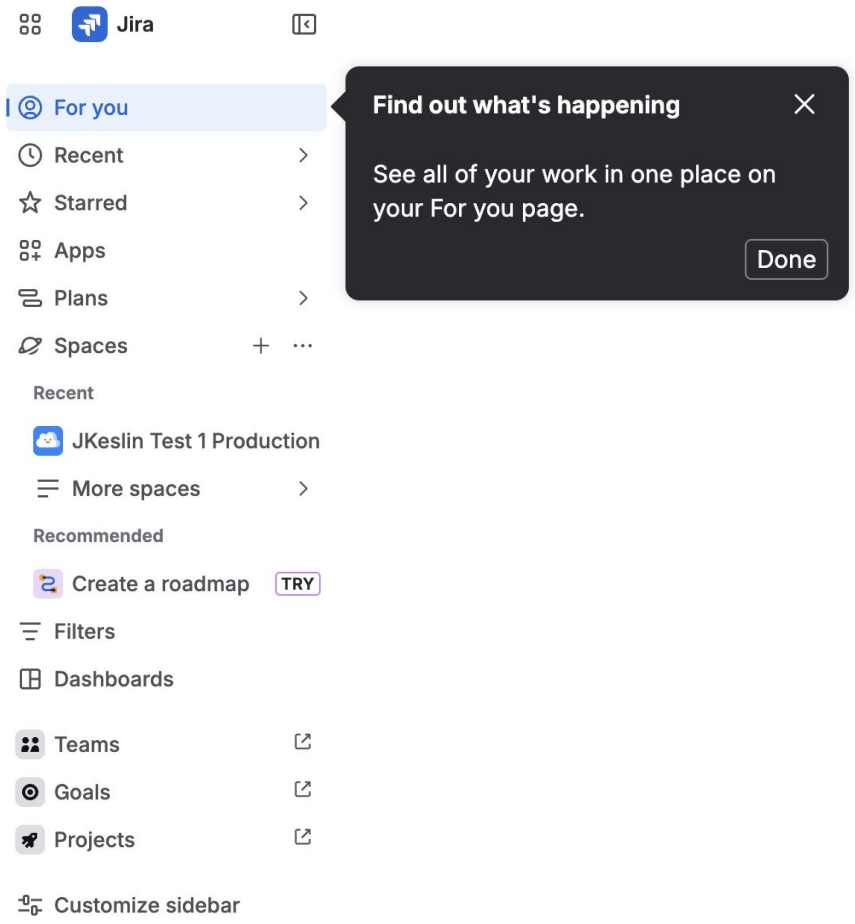
Optimize for the higher layers

Prioritize common use-cases
Make it easy to do typical things like setting text labels, colors, basic settings

Don’t forget complex but frequent things
Integrating analytics or spotlight tour elements shouldn’t require ejecting to hard-mode



The hard mode can be kinda hard
If a consumer really needs the added control, they’re usually willing to pay for it



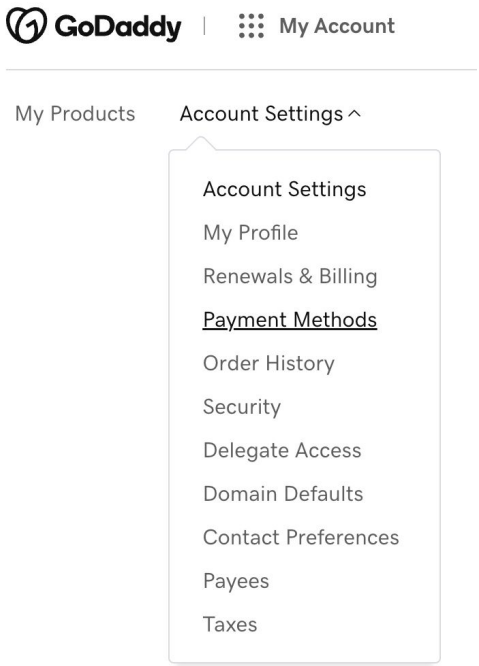

```
// For you item declaration
forYouItem: {
  url: "/for-you",
  ItemWrapper: WithForYouSpotlight
}
```

```
function WithForYouSpotlight({
  children
}) {
  return <PopoverProvider>
    <PopoverTarget>
      { children }
    </PopoverTarget>
    <PopoverContent ...>
      <ForYouSpotlightCard />
    </PopoverContent>
  </PopoverProvider>;
}
```

Principle 3:

Don’t build a **Homer Car**

The Simpsons, season 2 episode 15

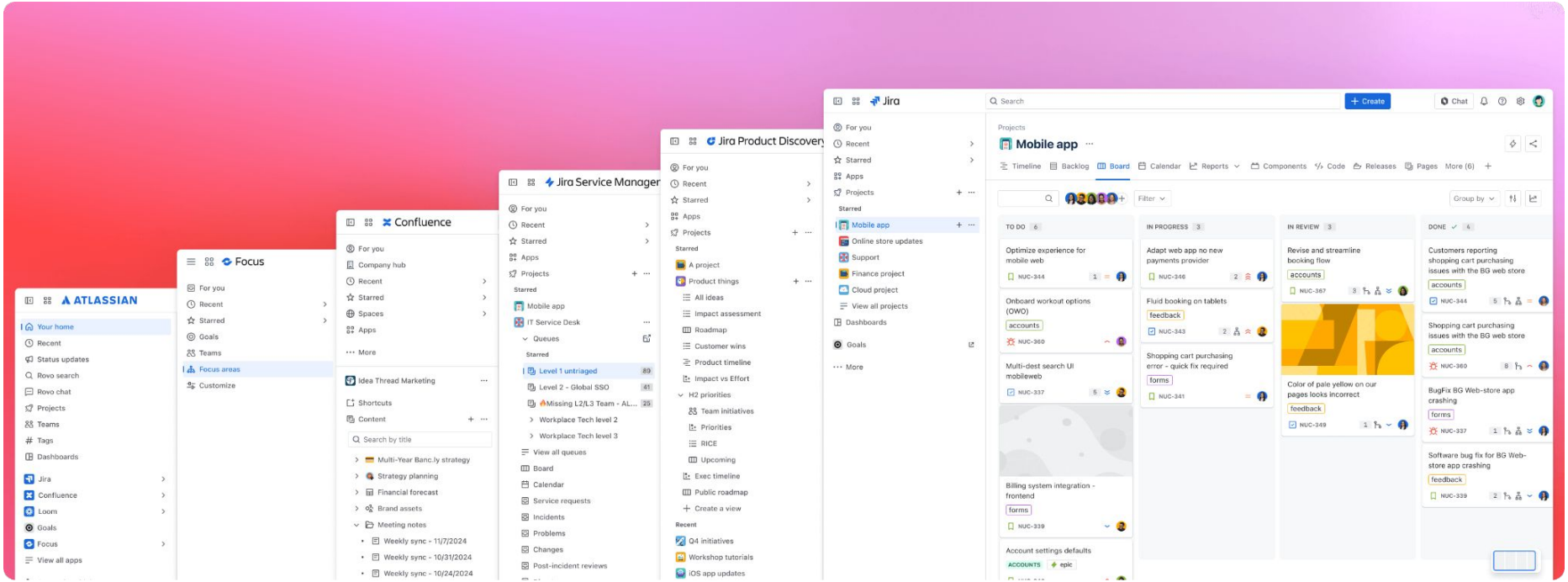

GoDaddy
My Account
My Products
Account Settings
Account Settings
My Profile
Renewals & Billing
Payment Methods
Order History
Security
Delegate Access
Domain Defaults
Contact Preferences
Payees
Taxes


Country/Region *
United States

Principle 4:

It’s okay to **be opinionated**

SHARED COMPONENTS BEYOND THE DESIGN SYSTEM



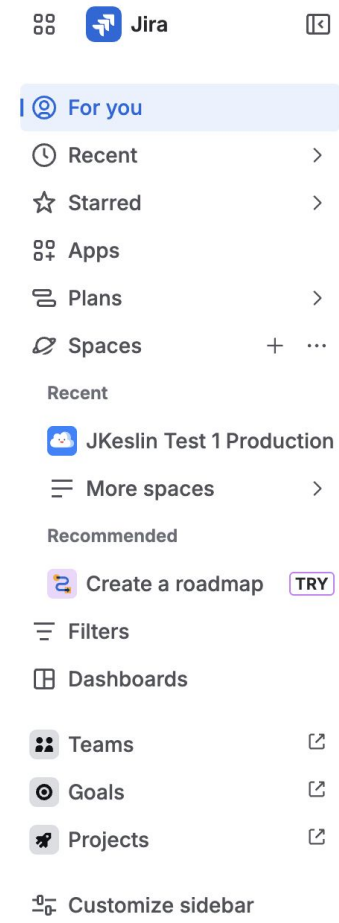
Principle 5:

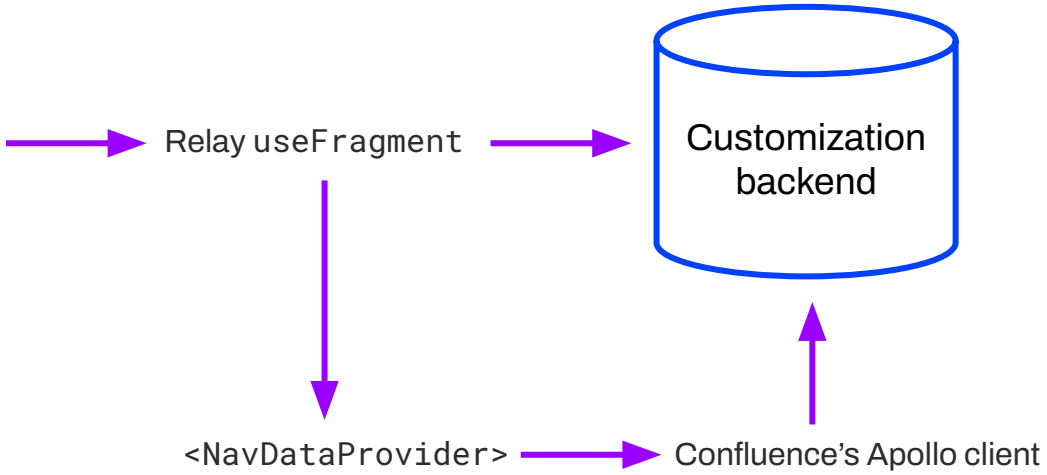
Consider including **batteries**

Reduce boilerplate as much as possible
Using your component shouldn't require writing a novel — keep it simple

Handle data fetching, if you can
Use mechanisms like Relay's fragments, for example, to encapsulate data requests

Keep an escape hatch, though
Despite your best efforts, sometimes a consumer needs more control


Jira
For you
Recent
Starred
Apps
Plans
Spaces
Recent
JKeslin Test 1 Production
More spaces
Recommended
Create a roadmap
TRY
Filters
Dashboards
Teams
Goals
Projects
Customize sidebar


Relay useFragment
Customization backend
<NavDataProvider>
Confluence's Apollo client

Principle 6:

Make it **maintainable**

Write thorough tests
Keep quality high — write plenty of unit and integration tests so your component is rock solid

Document liberally
You won't always be there for support — document everything

Keep a changelog
Updates happen — keep your consumers appraised of what changed when

Jira

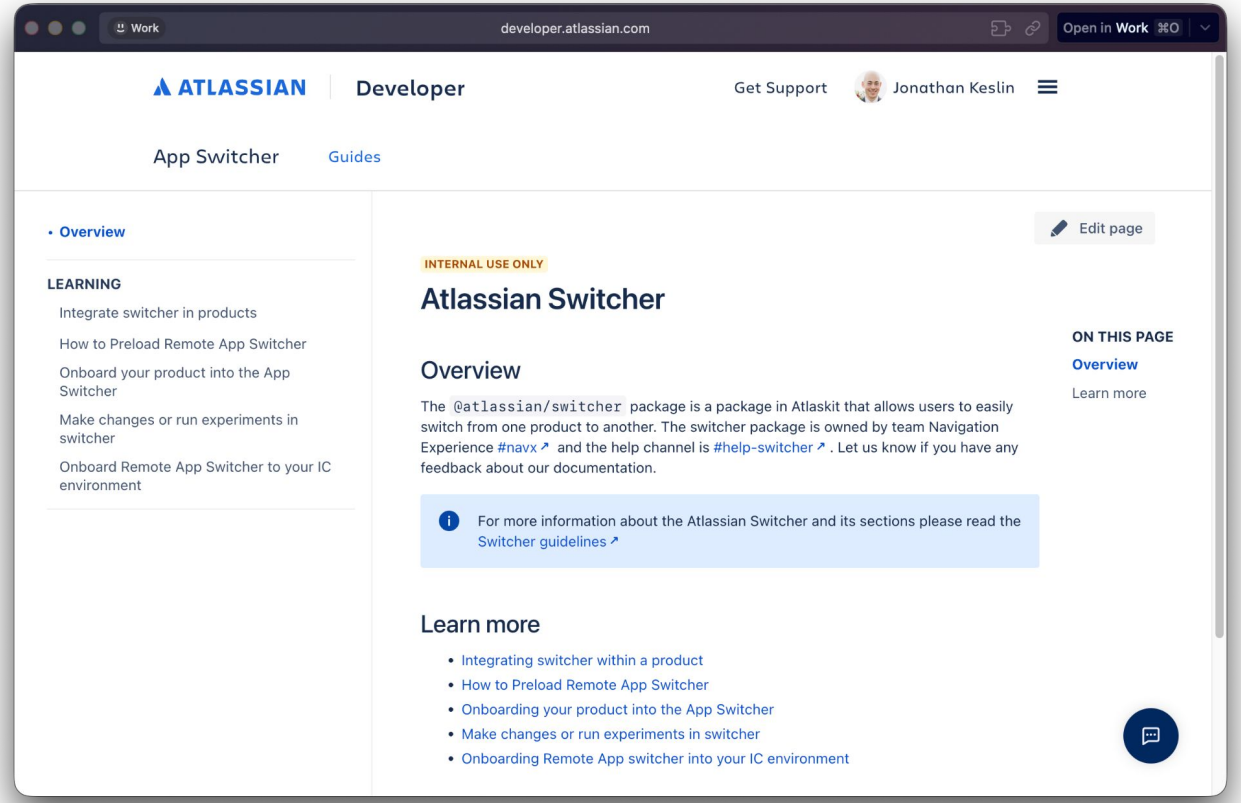
Te team-site

- Home
- Company hub
- Jira
- Confluence
- Loom
- Focus
- Talent
- ••• 15 more

Discover more apps

- More Atlassian apps

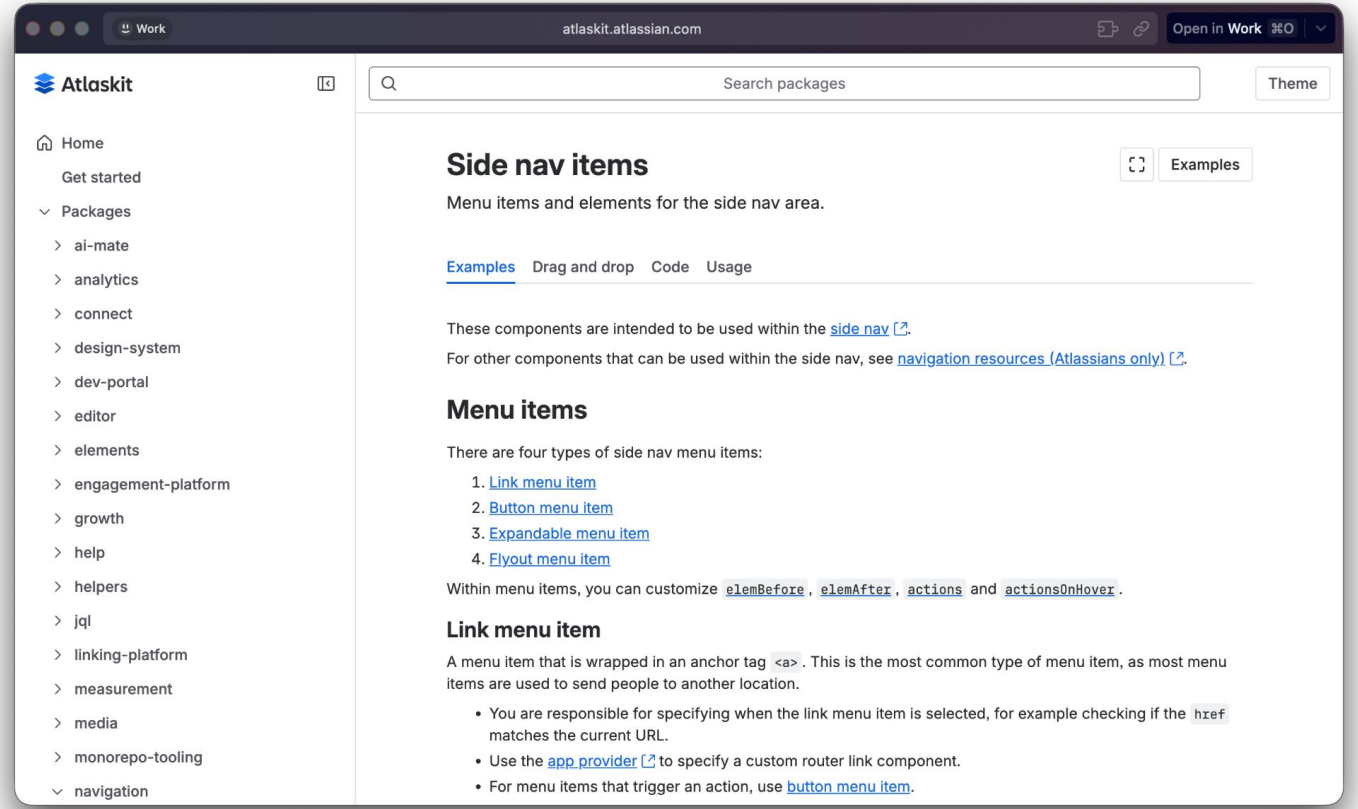
Manage list



Foster an **ecosystem**

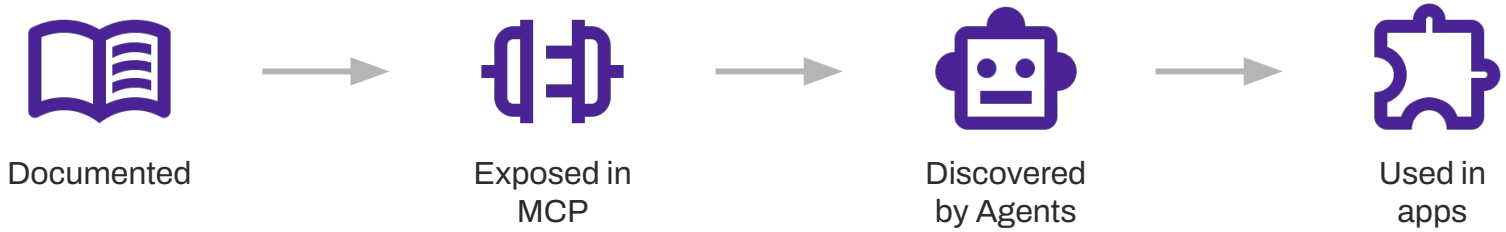
Shared components are super valuable
They de-duplicate effort, become a force multiplier, and improve consistency across apps

Discoverability makes a big difference
Investing in a common doc site / marketplace for shared components pays dividends



This is a screenshot from the public version of the site, not the internal one

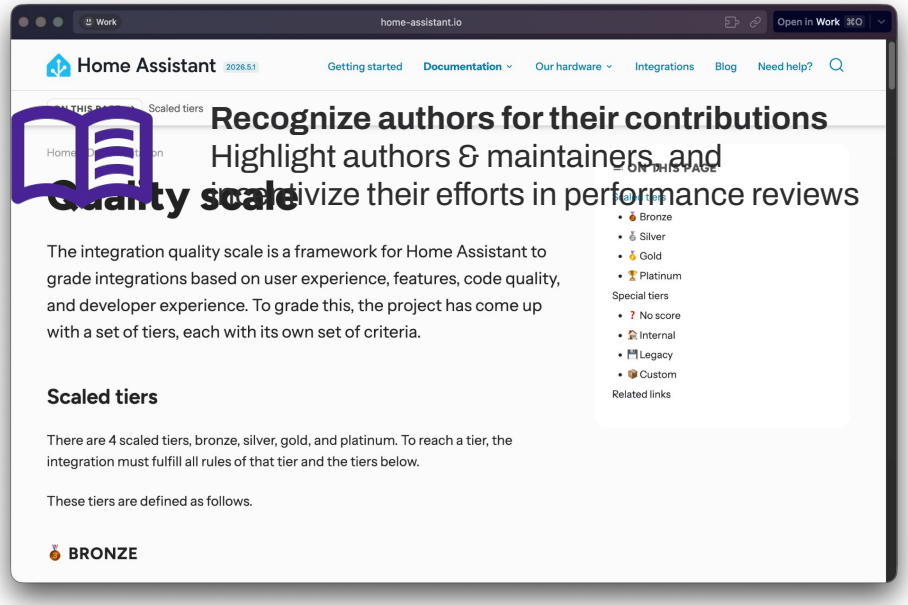
Good docs matter more than ever



Provide **governance**

Define patterns & practices
Give builders something to aim for as they build out their shared components

Recognize authors for their contributions
Highlight authors & maintainers, and incentivize their efforts in performance reviews



A short **summary**

Shared Components are important
The design system can’t and shouldn’t do everything

Build Shared Components thoughtfully
Allow them to evolve on their own and build them to a high standard

Foster a healthy Shared Component ecosystem
Thorough docs, strong patterns & practices, and recognition from leadership

Now, go build something **awesome!**

Thank you!

jonathankeslin.com/cascadiajs26